DevSecOps: quyền, secret và chuỗi cung ứng

Một chủ đề, một tài liệu: mô hình, bài tập, lỗi thường gặp và câu hỏi phỏng vấn.

Đọc theo thứ tự trong tập. Làm lab trên môi trường riêng, ghi kết quả và đối chiếu tài liệu chính thức theo phiên bản bạn đang dùng.

Bám theo bốn JD người học cung cấp | 11/10/2026

Tài liệu độc lập theo một chủ đề. Đọc mô hình trước, làm lab trên môi trường riêng, ghi bằng chứng và tự giải thích failure mode. Các ví dụ dựa trên ứng dụng Notes API và bốn JD đã cung cấp.

Tư duy rủi ro và quyền tối thiểu

Đừng bắt đầu bằng danh sách công cụ bảo mật. Hãy hỏi: cần bảo vệ tài sản nào, ai có thể truy cập, đường tấn công hoặc lỗi vận hành nào khả dĩ, hậu quả là gì và có thể phát hiện/giảm ảnh hưởng ra sao? Với API ghi chú, tài sản là dữ liệu ghi chú, secret database, quyền cloud, source code và pipeline. Một token CI có quyền xóa toàn bộ tài nguyên là rủi ro lớn hơn nhiều so với token chỉ được đẩy image vào registry.

Threat modeling đơn giản có thể làm trên sơ đồ request: khoanh vùng các điểm tin cậy thay đổi, như browser -> API, API -> database, CI -> cloud. Với mỗi mũi tên, hỏi authentication, authorization, encryption và log. Authentication xác nhận ai/cái gì đang gọi; authorization xác định được phép làm gì. Mã hóa đường truyền bằng TLS bảo vệ dữ liệu trên mạng nhưng không sửa quyền sai tại database. Mỗi cơ chế xử lý một lớp rủi ro khác.

Secrets gồm mật khẩu, API token, private key và credential cloud. Không đưa chúng vào Git, image, log hoặc ảnh chụp màn hình. Dùng hệ thống quản lý secret phù hợp, cấp quyền truy cập theo workload, luân chuyển khi cần và có quy trình thu hồi. Một secret bị commit rồi xóa ở commit mới vẫn có thể tồn tại trong lịch sử; phải coi nó đã lộ và rotate. Đừng tin vào việc chỉ thêm **.gitignore** sau khi lỡ commit.

Bài tập review rủi ro

Vẽ đường từ pull request -> CI runner -> image registry -> cluster -> database. Ghi quyền tối thiểu mỗi thành phần cần. Đánh dấu nơi secret đi qua và log nào có thể vô tình lộ nó. Chọn ba rủi ro có tác động cao nhất, một biện pháp giảm xác suất, một cách phát hiện và một cách phục hồi cho từng rủi ro.

Software supply chain và bảo vệ pipeline

Ứng dụng phụ thuộc thư viện, image gốc, build tools và workflow bên thứ ba. Supply chain security quan tâm nguồn gốc và tính toàn vẹn của các đầu vào/đầu ra đó. Khóa phiên bản dependency, xem cảnh báo lỗ hổng, cập nhật có kiểm tra, dùng image từ nguồn tin cậy và giữ bản ghi build/release. Quét lỗ hổng không phải phán quyết cuối: cần đánh giá liệu mã dễ bị ảnh hưởng có được dùng, bối cảnh tiếp xúc và khả năng vá.

CI thường nhận code từ pull request, nên dữ liệu do người ngoài sửa có thể đi vào shell script hoặc action. Workflow có quyền cao phải được thiết kế cẩn thận; không để code chưa tin cậy chạy cùng credential triển khai. Ưu tiên credential ngắn hạn theo cơ chế identity federation nếu nền tảng hỗ trợ, giới hạn quyền token và tách build/test khỏi deploy. Một pipeline chạy tự động nhưng không biết artifact nào được tạo từ commit nào khó kiểm soát hơn quy trình thủ công có bằng chứng.

Lab kiểm tra pipeline

Trên repository thử nghiệm, liệt kê mọi secret và quyền workflow cần. Không in giá trị secret. Xem action/container nào do bên thứ ba cung cấp, phiên bản nào đang dùng và cách cập nhật an toàn. Thêm một kiểm tra dependency hoặc image scan theo tài liệu chính thức của công cụ bạn chọn; đọc ít nhất một cảnh báo và ghi lý do ưu tiên/không ưu tiên xử lý. Không chặn toàn bộ build bằng ngưỡng tùy tiện khi chưa hiểu độ nhiễu.

Threat model có thể kiểm tra được

Thay vì ghi chung "bảo mật tốt", hãy mô tả một tình huống: người ngoài tạo pull request sửa workflow để in biến môi trường. Tài sản bị đe dọa là credential CI. Đường đi là code chưa tin cậy -> runner -> token/secret -> hệ thống cloud. Biện pháp phòng là giảm quyền token, tách job dùng credential khỏi PR chưa tin cậy và dùng chính sách approval theo nền tảng. Cách phát hiện có thể là review workflow, audit log và cảnh báo hành vi token lạ. Khi nghi lộ, thu hồi/rotate credential và xem hành động đã thực hiện bằng nó.

Tình huống thứ hai: ứng dụng ghi toàn bộ HTTP header vào log, vô tình có bearer token. Biện pháp phòng là chỉ log trường cần thiết và lọc thông tin nhạy cảm. Nếu đã lộ, xóa/giới hạn truy cập log theo quy trình, rotate token, xác định ai đã truy cập và đánh giá tác động. Tình huống thứ ba: database mở public để "debug cho nhanh". Biện pháp phòng là đường private, firewall hẹp và truy cập quản trị tạm thời có ghi nhận. Những ví dụ này minh họa một quy tắc: giảm cả xác suất lẫn phạm vi thiệt hại, rồi chuẩn bị khả năng phát hiện và phục hồi.

Bảng rủi ro tự lập

Tạo bốn cột: tài sản, tình huống, tác động, kiểm soát. Điền ít nhất năm dòng cho repo của bạn: source code, CI token, container image, database và backup. Sắp xếp ưu tiên theo tác động và khả năng xảy ra trong bối cảnh của bạn, không theo điểm số máy quét một cách máy móc. Với mỗi kiểm soát, ghi bằng chứng kiểm tra được, ví dụ policy CI, cấu hình network hoặc log restore.

Backup và restore ứng dụng ghi chú

Ứng dụng mẫu dùng SQLite để lab đơn giản. File **lesson/labs/notes-api/backup_lab.py** dùng API backup của SQLite để tạo bản sao nhất quán rồi phục hồi sang file **khác**. Trước khi chạy, tạo ít nhất một ghi chú và dừng ứng dụng để bài học dễ quan sát. Chạy **python backup_lab.py** trong thư mục ứng dụng. Script từ chối nếu file backup hoặc file restore cũ đã tồn tại để tránh ghi đè; kiểm tra chúng trước khi quyết định xóa dữ liệu lab. Kết quả mong đợi là số ghi chú ở bản gốc và bản restore bằng nhau.

So sánh số lượng chưa đủ để bảo đảm mọi dữ liệu đúng. Hãy mở file phục hồi bằng một lệnh Python hoặc cho API chạy với **NOTES_DB** trỏ tới **restored-notes.db**, rồi gọi **GET /notes** và đối chiếu nội dung. Dừng bản API cũ trước nếu cùng dùng port 8000. Ghi thời gian bắt đầu và thời gian người dùng có thể đọc lại dữ liệu. Đó là RTO quan sát trong lab; không tự nhận là RTO production. Tạo thêm ghi chú sau backup nhưng trước restore để thấy dữ liệu mới này không có trong bản backup; khoảng mất dữ liệu đó minh họa RPO.

Lưu ý: backup cùng nằm trên một đĩa với dữ liệu gốc sẽ không giúp khi đĩa mất. Trong hệ thống thật, cần nơi lưu riêng, kiểm soát quyền, mã hóa phù hợp, retention và diễn tập restore định kỳ. Một script local đơn giản dạy quy trình, không thay thế chiến lược backup production.

Bảo mật ứng dụng và tự động hóa có kiểm soát

JD 2 còn yêu cầu OWASP Top 10, threat modeling, SAST/DAST/SCA, CVE và least privilege. Với ví dụ ứng dụng tải file vào Blob Storage rồi phát message xử lý nền, tài sản gồm file người dùng, metadata, credential và token; điểm vào gồm API, Service Bus message, storage event và pipeline. Hỏi theo STRIDE: ai có thể giả mạo người dùng/message; dữ liệu có thể bị sửa; hành động có audit; file có bị lộ; hệ thống có thể bị nghẽn; quyền có bị leo thang. Với mỗi rủi ro, ghi một biện pháp và phép kiểm tra: giới hạn loại/kích thước file, token đúng audience, quét dependency, quyền Storage tối thiểu và log không chứa secret.

SAST tìm vấn đề trong source; SCA tìm dependency có lỗ hổng đã biết; DAST thử ứng dụng đang chạy. CVE cần triage theo phiên bản thực dùng, đường code, cấu hình, độ phơi bày và bản vá. Quy trình ghi mức ảnh hưởng, owner, deadline, mitigation và bằng chứng khi đóng. Pipeline phải quét lại sau khi nâng dependency/image.

Bài tập quyền tối thiểu

Liệt kê ba identity: developer đọc log, App Service đọc đúng secret Key Vault và ghi Blob, pipeline triển khai vào một resource group. Với từng identity, ghi scope nhỏ nhất, hành động cần, cách cấp token và cách thu hồi. Kiểm tra riêng quyền control plane Azure RBAC và quyền data plane của dịch vụ đích; thấy resource trên portal không luôn đồng nghĩa đọc được dữ liệu bên trong.

Threat model bắt đầu từ dữ liệu và ranh giới tin cậy

Với Notes API, tài sản cần bảo vệ là dữ liệu ghi chú, credential DB, quyền deploy và log có thể chứa dữ liệu người dùng. Vẽ đường Internet -> ingress -> API -> DB, thêm CI -> registry -> cluster và operator -> portal. Ở mỗi mũi tên hỏi: ai có thể gửi dữ liệu? ai xác thực? ai có quyền thay cấu hình? dữ liệu được mã hóa/chứng thực ở đoạn nào? log/audit nào giúp phát hiện lạm dụng? STRIDE là khung gợi ý: giả mạo danh tính, sửa dữ liệu, chối bỏ hành động, lộ thông tin, từ chối dịch vụ, nâng quyền. Không cần liệt kê mọi mối đe dọa tưởng tượng; ưu tiên đường có thể khai thác và tác động lớn.

Ví dụ pipeline có token cluster-admin. Một PR sửa pipeline để in token hoặc gọi URL ngoài có thể chiếm cluster nếu job chạy với secret. Biện pháp không chỉ là mask token trong log; cần hạn chế secret chỉ ở protected branch/environment, review thay đổi pipeline, runner tin cậy, token ngắn hạn/identity federation và scope tối thiểu. Tương tự, Kubernetes Secret base64 không phải encryption end-to-end; quyền đọc API, etcd encryption, backup, người có quyền exec vào Pod và log đều là ranh giới phải xét.

OWASP Top 10 là bản đồ rủi ro, không phải checklist đủ dùng

OWASP Top 10:2025 đã cập nhật các nhóm như Broken Access Control, Security Misconfiguration và Software Supply Chain Failures. Khi học, gắn mỗi nhóm với một ví dụ Notes API. **Broken Access Control:** user A đoán ID note của user B và API chỉ kiểm tra "đã đăng nhập". **Misconfiguration:** debug endpoint public hoặc CORS mở quá rộng. **Injection:** ghép string SQL từ input. **Supply chain:** dependency/image base chứa mã bị khai thác hoặc pipeline lấy package không đáng tin. Với mỗi ví dụ, ghi phép thử và biện pháp ở code, CI, runtime và monitoring. Đừng kết luận "đã scan là an toàn"; scan chỉ tìm tập lỗi mà scanner biết.

SAST phân tích code không cần app chạy; DAST kiểm thử app đang chạy; SCA theo dõi dependency; image scan quan sát package trong image. CVE có mức nghiêm trọng nhưng cần triage: version thực, package có dùng không, đường tấn công có mở không, exploit đang có, giảm thiểu hiện tại và deadline vá. Không trì hoãn vô hạn bằng "không dùng code path" nếu chưa có bằng chứng. Ghi exception có owner, ngày hết hạn và điều kiện kiểm tra lại.

Lab threat model trên giấy có lời giải

Tài liệu đầu vào: sơ đồ Notes API với HTTPS ingress, API, DB, registry, CI và secret manager. **Bước 1:** chọn ba tài sản: note content, DB credential, deployment permission. **Bước 2:** đánh dấu ba trust boundary: Internet/edge, CI/production, app/DB. **Bước 3:** viết một attack path cho mỗi tài sản. Ví dụ attacker lấy token CI qua job PR -> sửa manifest -> deploy image độc; user A gọi **GET /notes/<id-B>** -> đọc note B; lỗi log in toàn bộ Authorization header -> lộ token qua hệ thống logging. **Bước 4:** cho mỗi path, chọn prevention, detection và recovery. **Bước 5:** ghi residual risk mà biện pháp chưa xử lý.

Lời giải hướng dẫn: token CI cần permission hẹp, secret chỉ trên trusted ref, review pipeline, audit và rotation khi lộ. IDOR cần authorization theo object ở API cùng test user A/B; WAF không thay kiểm tra quyền. Log token cần redaction ở source/collector, hạn chế quyền log, retention và quy trình thu hồi token đã lộ. Nếu chỉ nói "dùng Vault" mà không chỉ ra workload nào được lấy secret nào, threat model chưa đủ cụ thể.

Hardening Linux/container/Kubernetes theo nguyên tắc

Giảm bề mặt tấn công: tắt service không cần, cập nhật có kế hoạch, firewall mặc định hạn chế, chạy app non-root, filesystem chỉ đọc khi hợp lý, drop Linux capabilities, không privileged container, giới hạn network egress/ingress, ServiceAccount quyền tối thiểu, image digest đã duyệt. Mỗi biện pháp phải có test chức năng: app cần ghi tmp ở đâu? DNS có bị NetworkPolicy chặn không? Container non-root có quyền đọc cert/DB file không? Hardening không thể chỉ là bật tất cả option rồi giao outage cho đội vận hành.

Secrets có vòng đời: tạo, cấp, dùng, rotate, thu hồi, audit. Một secret đã vào Git history không được coi là an toàn chỉ vì commit mới xóa nó; cần revoke/rotate. Backup có thể chứa secret cũ nên chính sách truy cập/retention cũng phải phù hợp. Với Key Vault/Vault, identity của workload là điểm quyết định; một app không nên đọc mọi secret trong môi trường.

Ca ứng phó: phát hiện credential registry trong log CI

Xác định token thuộc principal nào, phạm vi quyền và thời gian log công khai; thu hồi/rotate theo quy trình, kiểm tra audit registry để xem có push/pull lạ, xác định image digest nào bị tác động, tạm dừng promotion nếu cần. Sau đó sửa pipeline để không in token và giảm scope/TTL, kiểm tra branch protection và secret exposure. Chỉ xóa log không thu hồi token là chưa xử lý rủi ro. Ghi incident và thông báo theo chính sách tổ chức; tránh sao chép token vào ticket điều tra.

Câu hỏi: khi nào một CVE critical chưa phải việc sửa đầu tiên? Khi không ảnh hưởng asset/runtime thực hoặc đã có mitigating control, trong khi một lỗ hổng bị khai thác trên đường public cần xử lý ngay; quyết định phải có bằng chứng và hạn chót. **Câu hỏi:** TLS có bảo vệ dữ liệu trong DB backup không? Không tự động; TLS bảo vệ đoạn truyền, backup cần mã hóa/quyền riêng. Nguồn: OWASP Top 10:2025, NIST SSDF.

Bảo vệ đường build và artifact

Giả sử developer merge code an toàn nhưng pipeline tải một package từ registry public bằng version range rộng. Một bản package mới độc hại có thể vào image mà source repo không đổi. Khóa dependency, dùng registry/allowlist phù hợp, tạo SBOM, scan và lưu provenance/digest giúp phát hiện và tái hiện. Nhưng lockfile không tự bảo vệ nếu attacker sửa lockfile qua PR được duyệt kém; reviewer phải hiểu thay đổi dependency và pipeline. Promotion nên dùng cùng artifact bất biến từ staging đến production, không rebuild với dependency khác.

Một image digest được ký hoặc có attestation tăng niềm tin về nguồn/build theo policy, nhưng phải kiểm tra ai giữ key/identity ký, build runner có đáng tin không và admission có thực thi policy không. “Có chữ ký” từ một pipeline đã bị chiếm không chứng minh code lành. Bảo mật supply chain là nhiều lớp: quyền source, runner, dependency, registry, deployment và runtime.

CVE triage bằng hai ví dụ

Scanner báo CVE critical ở thư viện A trong image, nhưng thư viện chỉ dùng bởi công cụ build và không có trong runtime image. Xác minh SBOM/runtime contents trước khi quyết định; có thể không phơi lộ production nhưng vẫn cần vá pipeline/build. Scanner báo CVE medium ở endpoint public có exploit hoạt động; ưu tiên có thể cao hơn. Ma trận triage nên ghi severity, exploitability, exposure, asset impact, compensating control, owner và deadline. Không “ignore forever” bằng một comment thiếu bằng chứng.

Phân quyền Kubernetes theo hành động

App Notes API cần đọc ConfigMap/Secret của chính nó khi start? Nhiều app nhận env/mount do kubelet cấp, không cần token gọi Kubernetes API trực tiếp. Nếu không dùng API, tắt automount ServiceAccount token theo chính sách để giảm bề mặt. Argo CD controller cần tạo Deployment/Service ở namespace được cấp, nhưng không nhất thiết được sửa mọi cluster-scoped resource. **kubectl auth can-i** và review RoleBinding/ClusterRoleBinding giúp kiểm tra quyền cụ thể; một quyền wildcard ***** trên ***** là tín hiệu phải giải thích. NetworkPolicy default-deny cần rule DNS/DB/telemetry thật, test từ Pod và theo dõi lỗi.

Ca lộ secret trong Git

Thứ tự: thu hồi/rotate secret thật, kiểm tra audit và tác động, chặn pipeline/deploy phụ thuộc secret cũ theo kế hoạch, rồi làm sạch lịch sử theo quy trình nếu cần. Không chỉ xóa dòng ở commit mới. Sau đó thêm secret scanning/gate, hướng dẫn dev dùng secret manager/local env an toàn, và kiểm tra backup/fork/cache chứa bản cũ. Trong incident report ghi thời gian phơi lộ và quyền của credential, không sao chép giá trị secret.

Bạn đạt bài khi viết được threat model có ba attack path thực, chọn controls ở đúng lớp và giải thích vì sao scan/secret manager không phải "nút an toàn". Nguồn: OWASP Top 10:2025, NIST SSDF.

Tự đánh giá

- Giải thích mô hình bằng lời của bạn, không dựa vào tên công cụ.
- Làm lab và lưu kết quả mong đợi cùng kết quả thực tế.
- Tạo một lỗi có kiểm soát, dùng bằng chứng để khoanh vùng và phục hồi.
- Trả lời một câu hỏi thiết kế: tại sao chọn phương án này, tradeoff và giới hạn là gì?

Nguồn chính thức và đọc thêm

- https://owasp.org/www-project-top-ten/
- https://csrc.nist.gov/pubs/sp/800/218/final
- https://developer.hashicorp.com/vault/docs